

C# - PHẦN 9.2

MULTICAST DELEGATE

Tài liệu học + ví dụ + bài tập thực hành

Nối tiếp Phần 9.1 - Delegate cơ bản

Ý tưởng trung tâm: Một biến Delegate có thể tham chiếu đến nhiều method cùng lúc. Khi gọi Delegate, các method được thực thi lần lượt.

1. Từ Delegate cơ bản đến Multicast Delegate

Ở bài 9.1, bạn đã có một Delegate tham chiếu đến một method:

```
CalculateDelegate operation = Calculator.Add;
```

Có thể hình dung:

```
operation
  ↓
Calculator.Add
```

Multicast Delegate mở rộng ý tưởng này:

```
Delegate
├── Method1
├── Method2
└── Method3
```

Khi gọi biến Delegate, các method trong danh sách sẽ được gọi lần lượt.

2. Ví dụ đơn giản nhất

Khai báo một Delegate dành cho thông báo:

```
public delegate void NotificationDelegate(string message);
```

Delegate này có thể tham chiếu đến các method có chữ ký:

```
void Method(string message)
```

Ví dụ ba method phù hợp:

```
public static void SendEmail(string message)
{
    Console.WriteLine($"Email: {message}");
}

public static void SendSms(string message)
{
    Console.WriteLine($"SMS: {message}");
}

public static void WriteLog(string message)
{
    Console.WriteLine($"Log: {message}");
}
```

Cả ba method đều nhận một string và trả về void, nên đều phù hợp với NotificationDelegate.

3. Thêm nhiều method bằng +=

Bắt đầu với một method:

```
NotificationDelegate notification = SendEmail;
```

Sau đó thêm method thứ hai và thứ ba:

```
notification += SendSms;  
notification += WriteLog;
```

Lúc này có thể hình dung:

```
notification  
├── SendEmail  
├── SendSms  
└── WriteLog
```

Khi gọi:

```
notification("Order completed");
```

Kết quả:

```
Email: Order completed  
SMS: Order completed  
Log: Order completed
```

Một lần gọi Delegate nhưng nhiều method cùng được thực thi.

4. Ví dụ đầy đủ

```
public delegate void NotificationDelegate(string message);  
  
public class Program  
{  
    public static void SendEmail(string message)  
    {  
        Console.WriteLine($"Email: {message}");  
    }  
  
    public static void SendSms(string message)  
    {  
        Console.WriteLine($"SMS: {message}");  
    }  
  
    public static void WriteLog(string message)  
    {  
        Console.WriteLine($"Log: {message}");  
    }  
  
    public static void Main(string[] args)  
    {  
        NotificationDelegate notification = SendEmail;  
  
        notification += SendSms;  
        notification += WriteLog;
```

```

        notification("Order completed");
    }
}

```

5. += có nghĩa gì với Delegate?

Đừng hiểu += ở đây giống phép cộng số học. Với Delegate:

```
notification += SendSms;
```

có nghĩa là: thêm SendSms vào danh sách các method mà notification sẽ gọi.

Danh sách các method này thường được gọi là Invocation List. Ở mức hiện tại, chỉ cần hiểu đó là danh sách method được lưu trong Delegate.

6. Thứ tự thực thi

Giả sử bạn thêm các method theo thứ tự:

```

NotificationDelegate notification = SendEmail;
notification += SendSms;
notification += WriteLog;

```

Khi gọi Delegate, thứ tự thực thi là:

1. SendEmail
2. SendSms
3. WriteLog

Tức là các method được gọi theo thứ tự đã được thêm vào Delegate.

7. Xóa method bằng -=

Có thể xóa một method khỏi Delegate bằng -=:

```
notification -= SendSms;
```

Trước khi xóa:

```

SendEmail
SendSms
WriteLog

```

Sau khi xóa SendSms:

```

SendEmail
WriteLog

```

Gọi lại:

```
notification("Payment completed");
```

Kết quả chỉ còn Email và Log.

8. Phân biệt =, += và -=

Toán tử	Ý nghĩa với Delegate
=	Gán hoặc thay thế danh sách hiện tại bằng method mới

<code>+=</code>	Thêm method vào danh sách
<code>-=</code>	Xóa method khỏi danh sách

Ví dụ:

```
NotificationDelegate notification = SendEmail;
notification += SendSms;

// Danh sách hiện tại: SendEmail, SendSms

notification = WriteLog;

// Danh sách cũ bị thay thế, chỉ còn WriteLog
```

9. Multicast Delegate có giá trị trả về

Delegate vẫn có thể trả về giá trị, nhưng cần chú ý khi nó chứa nhiều method.

```
public delegate int CalculateDelegate(int a, int b);

public static int Add(int a, int b)
{
    return a + b;
}

public static int Multiply(int a, int b)
{
    return a * b;
}
```

Nếu:

```
CalculateDelegate operation = Add;
operation += Multiply;

int result = operation(2, 3);
```

Cả Add và Multiply đều được gọi, nhưng result nhận giá trị trả về của method cuối cùng trong Invocation List. Ở ví dụ này, result là 6 từ Multiply(2, 3).

Vì vậy khi mới học Multicast Delegate, dùng Delegate trả về void sẽ dễ hiểu và ít gây nhầm lẫn hơn.

10. Liên hệ với bài 9.1

Ở 9.1:

```
CalculateDelegate operation = Calculator.Add;
```

Delegate đang tham chiếu đến một method.

Ở 9.2:

```
NotificationDelegate notification = SendEmail;
notification += SendSms;
notification += WriteLog;
```

Delegate đang chứa nhiều method. Thực tế Delegate trong C# vốn hỗ trợ multicast; bài 9.2 chỉ tập trung vào khả năng này.

11. Ví dụ gần với hệ thống .NET

Giả sử sau khi tạo một yêu cầu hỗ trợ sinh viên, hệ thống cần gửi email, ghi log và hiển thị thông báo.

```
public delegate void RequestCreatedHandler(string requestId);
public static void SendEmail(string requestId)
{
    Console.WriteLine($"Email sent for request {requestId}");
}

public static void WriteLog(string requestId)
{
    Console.WriteLine($"Request {requestId} was written to log.");
}

public static void ShowNotification(string requestId)
{
    Console.WriteLine($"Request {requestId} was created successfully.");
}

RequestCreatedHandler handler = SendEmail;
handler += WriteLog;
handler += ShowNotification;

handler("YC01");
```

Ý tưởng này sẽ là nền tảng để học Event ở các bài sau. Hiện tại chưa cần hiểu Event.

12. Nếu một method ném Exception thì sao?

Giả sử Delegate chứa:

```
SendEmail
SendSms
WriteLog
```

Nếu SendSms ném exception và exception không được xử lý, luồng gọi sẽ bị gián đoạn. Method phía sau có thể không được chạy.

Ở bài 9.2, chỉ cần nhận biết điều này; cách xử lý nâng cao sẽ học ở phần Exception nâng cao.

13. Tóm tắt kiến thức cần nhớ

- Delegate có thể tham chiếu đến nhiều method có chữ ký phù hợp.
- Dùng += để thêm method.
- Dùng -= để xóa method.
- Dùng = để gán hoặc thay thế danh sách method hiện tại.
- Khi gọi Delegate, các method được thực thi theo thứ tự đã thêm.
- Với Multicast Delegate có return value, lời gọi trực tiếp nhận giá trị trả về của method cuối cùng.

Công thức ghi nhớ: = thay thế | += thêm | -= xóa

14. Bài tập 9.2 - Multicast Delegate

Mục tiêu: luyện cách tạo Multicast Delegate, thêm/xóa method và quan sát thứ tự thực thi.

Cấu trúc project gợi ý:

```
MulticastDelegateBasic/  
├── NotificationDelegate.cs  
├── NotificationService.cs  
└── Program.cs
```

A. NotificationDelegate.cs

Tự khai báo một Delegate có yêu cầu:

- Tên: NotificationDelegate
- Nhận: string message
- Trả về: void

B. NotificationService.cs

Tạo static class NotificationService và ba method:

- SendEmail
- SendSms
- WriteLog

Cả ba method đều nhận string message và trả về void. Mỗi method in một thông báo khác nhau, ví dụ:

```
Email: Order completed  
SMS: Order completed  
Log: Order completed
```

C. Program.cs

Thực hiện đúng thứ tự sau:

4. Tạo biến NotificationDelegate trỏ tới SendEmail.
5. Dùng += thêm SendSms.
6. Dùng += thêm WriteLog.
7. Gọi Delegate với chuỗi "Order completed".
8. Dùng -= xóa SendSms.
9. Gọi Delegate lại với chuỗi "Payment completed".

Kết quả lần đầu phải có:

```
Email  
SMS  
Log
```

Kết quả lần hai chỉ còn:

```
Email  
Log
```

D. Câu hỏi tự kiểm tra

10. Multicast Delegate khác Delegate cơ bản ở đâu?
11. += có tác dụng gì với Delegate?
12. -= có tác dụng gì?
13. Nếu dùng = thay vì += thì chuyện gì xảy ra?

15. Mức cần đạt trước khi qua bài tiếp theo

Bạn chưa cần học Action, Func, Predicate hay Event trong bài này. Chỉ cần chắc:

- Tạo được một Delegate có chữ ký phù hợp.
- Dùng =, += và -= đúng ý nghĩa.
- Biết một Delegate có thể gọi nhiều method.
- Biết thứ tự gọi theo thứ tự thêm method.
- Giải thích được vì sao dùng void sẽ dễ hiểu hơn khi học Multicast Delegate.